

Practice Set: Default Arguments in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define what a **Default Argument** is in C++. Describe the three primary reasons for using default arguments (API flexibility, code duplication reduction, and backward compatibility).

Question 2: Explain how default arguments work in memory. Does the compiler handle substitution via runtime branching or compile-time instruction generation, and how does the Call Stack behave?

Question 3: State the declaration syntax for a function with default arguments. Explain **Rule 1: The Right-to-Left Trailing Order** and why violations cause compile-time errors.

Question 4: Identify which of the following function declarations are **VALID** and which are **INVALID** according to the right-to-left rule:

- a) `int add(int x, int y, int z = 0);`
- b) `int add(int x = 0, int y, int z);`
- c) `int add(int x, int y = 0, int z = 0);`
- d) `int add(int x, int y = 0, int z);`
- e) `int add(int x = 0, int y = 0, int z = 0);`

Question 5: Explain **Rule 2: Declaration vs. Definition**. Why should default values be specified only in the function declaration (prototype) rather than both declaration and definition?

Question 6: Explain **Rule 3: Positional Argument Mapping**. Can a caller skip an intermediate default parameter to provide a value for a later trailing parameter?

Question 7: Compare **Default Arguments** with **Function Overloading** across four criteria: Number of Function Bodies, Compile-Time Mechanism, Code Maintenance, and Architectural Flexibility.

Question 8: Explain the common gotcha regarding **Overloading Ambiguity** when default arguments are combined with function overloading (e.g., `void print(int x);` and `void print(int x, int y = 10);`).

Question 9: Where is the recommended architectural location to specify default arguments in a modular multi-file C++ project, and why?

Question 10: Write a complete C++ program that implements a function `calculateInterest(double principal, double rate = 5.0, int time = 1)` demonstrating:

Calling with only the principal (using two defaults).

Overriding the rate while using the default time.

Overriding all arguments.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Does using default arguments introduce any runtime CPU execution overhead compared to passing arguments explicitly?

Question 12 (Viva): What compile-time error occurs if default values are repeated in both the function prototype and the function definition header?

Question 13 (Viva): Why is a declaration like `void display(int a = 1, int b, int c = 3);` rejected by the C++ compiler?

Question 14 (Viva): Can default arguments be specified for all parameters of a function (e.g., `int compute(int a = 0, int b = 0);`)?

Question 15 (Viva): How does the compiler resolve a function call like `add(5, 6)` when the function is declared as `int add(int x, int y, int z = 0)`?

Question 16 (Viva): If a function is declared as `void test(int a, int b = 20, int c = 30);`, is the call `test(5, , 50);` valid in C++?

Question 17 (Viva): Why does calling `print(5)` cause a compilation failure when both `void print(int x)` and `void print(int x, int y = 10)` are declared in the same scope?

Question 18 (Viva): Where in the Call Stack (Activation Record) are default argument values placed during runtime execution?

Question 19 (Viva): How do default arguments support backward compatibility in software libraries?

Question 20 (Viva): During which phase of compilation are missing default arguments inserted into the generated assembly code?

Answers & Explanations

Answer: 01

Definition: A Default Argument is a fallback value assigned to a formal parameter in the function declaration. If the caller does not supply an actual argument for that parameter, the compiler automatically substitutes the default value.

Reasons to Use:

Function Call Flexibility: Allows calling the same function with varying numbers of arguments.

Reduces Code Duplication: Avoids writing multiple overloaded functions that just forward constant values.

Backward Compatibility: Allows adding new parameters to existing functions without breaking existing call sites in legacy code.

Answer: 02

Memory & Compilation: Substitution occurs entirely at compile time with zero runtime branching overhead. When the compiler detects missing trailing arguments, it inspects the prototype and inserts the constant default value directly into the generated assembly code.

Call Stack: At runtime, the default value is pushed onto the function's Stack Frame (Activation Record) alongside the explicitly passed arguments.

Answer: 03

Syntax: `return_type function_name(type1 param1, type2 param2 = default_value);`

Rule 1 (Right-to-Left Trailing Order): Default arguments must always be assigned from right to left. Once a parameter receives a default value, all parameters to its right must also have default values. If non-default parameters follow a default parameter, positional binding cannot determine which parameter was omitted.

Answer: 04

- a) `int add(int x, int y, int z = 0);` — VALID** (Rightmost parameter has default).
- b) `int add(int x = 0, int y, int z);` — INVALID** (Non-default parameters y and z follow default parameter x).
- c) `int add(int x, int y = 0, int z = 0);` — VALID** (Trailing two parameters have defaults).
- d) `int add(int x, int y = 0, int z);` — INVALID** (Non-default parameter z follows default parameter y).
- e) `int add(int x = 0, int y = 0, int z = 0);` — VALID** (All parameters have defaults).

Answer: 05

Default argument values must be specified once in the function declaration (prototype). Repeating default values in both the declaration and definition causes a compilation error ("redefinition of default argument").

Answer: 06

Positional Argument Mapping: Arguments are matched strictly by position from left to right.

Skipping Parameters: You cannot skip intermediate parameters. To supply a custom value for a third parameter, explicit arguments must be provided for all preceding parameters.

Answer: 07

Feature	Default Arguments	Function Overloading
Function Bodies	Single function definition handles multiple call patterns.	Requires writing distinct function definitions for each signature.
Compile-Time Mechanism	Compiler fills in missing constants at call sites.	Compiler resolves matching function via name mangling

Feature	Default Arguments	Function Overloading
		and signatures.
Code Maintenance	High; changes to algorithm logic are made in one place.	Lower; logic must be maintained across multiple function bodies.
Flexibility	Ideal when omitted arguments have logical fallback constants.	Ideal when functions implement different algorithms or data types.

Answer: 08

Declaring `void print(int x);` alongside `void print(int x, int y = 10);` causes an ambiguity error when invoking `print(5)`. The compiler cannot determine whether to call the single-parameter version or the two-parameter version with `y` defaulted.

Answer: 09

In public header files (`.h/.hpp`) inside the function prototype declaration. This makes default values visible to any translation unit including the header file.

Answer: 10

```
#include <iostream>
using namespace std;
// Function prototype specifying default values
double calculateInterest(double principal, double rate = 5.0, int time = 1);
int main()
{
    // 1. Uses default rate (5.0) and default time (1)
    cout << "Interest 1: $" << calculateInterest(1000.0) << endl;

    // 2. Overrides rate (7.5), uses default time (1)
    cout << "Interest 2: $" << calculateInterest(1000.0, 7.5) << endl;

    // 3. Overrides both rate (8.0) and time (3)
    cout << "Interest 3: $" << calculateInterest(1000.0, 8.0, 3) << endl;

    return 0;
}

// Function definition (defaults are not repeated here)
double calculateInterest(double principal, double rate, int time)
{
    return (principal * rate * time) / 100.0;
}
```

}

Answer: 11

No runtime CPU overhead is introduced because parameter substitution occurs entirely at compile time.

Answer: 12

A compilation error stating "redefinition of default argument".

Answer: 13

Because it violates the right-to-left trailing order rule; non-default parameter b follows default parameter a.

Answer: 14

Yes, all parameters can have defaults, allowing the function to be called with zero arguments.

Answer: 15

The compiler translates the call to `add(5, 6, 0)` by inserting the default constant 0 for `z` into the generated assembly code.

Answer: 16

No, C++ does not support blank positional gaps; arguments must be supplied continuously from left to right.

Answer: 17

It results in a compile-time error ("call to 'print' is ambiguous") because both functions match the invocation.

Answer: 18

They are pushed into the function's Stack Frame (Activation Record) alongside the other parameters.

Answer: 19

They allow new parameters to be added to existing functions with default fallback values without modifying or breaking existing legacy call sites.

Answer: 20

During the **compilation phase** when generating assembly code instructions.